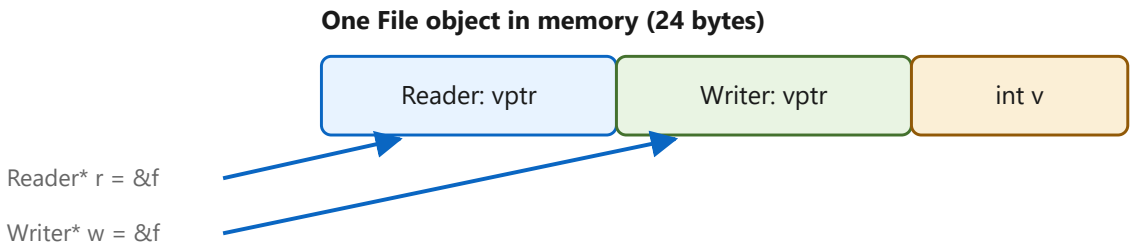
# The Object Model in Depth

What lies beneath inheritance: multiple inheritance and object layout, the diamond and virtual inheritance, RTTI and private inheritance. Builds on Object-Oriented Programming.

## Multiple inheritance and object layout

A class may have several bases. The sensible use is **several pure interfaces** (abstract classes with no data), as in Java:

```
struct Reader { virtual int read() = 0;          virtual ~Reader() = default;
};
struct Writer { virtual void write(int) = 0;     virtual ~Writer() = default;
};
struct File : Reader, Writer {                   // implements both interfaces
    int v = 0;
    int read() override { return v; }
    void write(int x) override { v = x; }
};
```



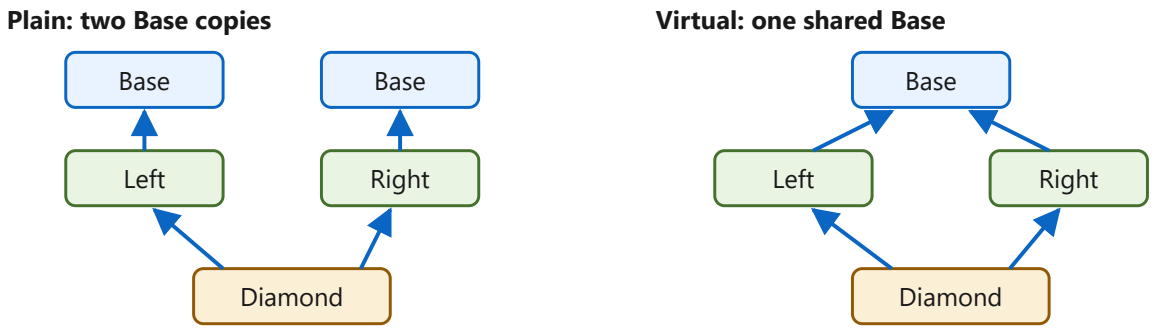

Converting File* to Writer* adjusts the address: the two pointers differ (compiled).

Each polymorphic base brings its own vptr, so the pointer value changes when you convert to the second base.

- The object is the bases laid out one after another, then the derived members. Compiled: `sizeof(File)` is **24** bytes (two vptrs and an `int`).
- Converting `File*` to the **second** base adjusts the pointer value (`static_cast` and implicit conversion do it for you). That is why you must not cast with `reinterpret_cast` or a C cast.
- Both bases are polymorphic, so `delete` through either needs a **virtual destructor**.
- **Name clashes:** if both bases have a member `f`, calling `obj.f()` is ambiguous. Qualify it (`obj.Reader::f()`) or override it in the derived class.
- Multiple inheritance of *implementation* (bases with data) is where trouble starts: the diamond.

---

## The diamond problem and virtual inheritance




Plain: d.id is ambiguous (two ids). Virtual: one id.

Virtual inheritance makes the shared base appear once.

```
struct Base  { int id; };
struct Left  : virtual Base {};          // "virtual" goes on the MIDDLE
classes
struct Right : virtual Base {};
struct Diamond : Left, Right {};         // one Base inside; d.id is not
ambiguous
```

| | PLAIN INHERITANCE | VIRTUAL INHERITANCE |
|---|---|---|
| Base subobjects in `Diamond` | two (compiled `sizeof` 8: two ids) | one (compiled `sizeof` 24: one id plus the machinery) |
| `d.id` | ambiguous: write `d.Left::id` | fine |
| Constructor order (compiled) | `Base Left Base Right Diamond`: Base built twice | `VBase VLeft VRight VDiamond`: shared base once, **first** |
| Who initializes the base? | each middle class | the **most derived** class, directly |
| Cost | none | extra pointer(s) per object, indirect access to the base, slower casts |

- The standard library's `std::iostream` is the classic use: `istream` and `ostream` inherit `ios` virtually, so `iostream` has one stream state.
- **Advice:** avoid diamonds. Use interfaces without data, or composition. When you need virtual inheritance, put the data in one place and let the most derived class initialize the base.

KEY POINT "What is the diamond problem and how do you solve it?" Two paths to one base duplicate its members and make access ambiguous. `virtual` inheritance shares one copy; the most derived class constructs it first. It costs memory and indirection, so prefer interfaces and composition.

## RTTI: `typeid` and `dynamic_cast`

**RTTI** (run-time type information) lets you ask for the *dynamic* type of an object behind a base pointer. It works only for **polymorphic** classes (at least one virtual function), because the type information lives next to the vtable.

```
Animal* a = new Dog;
typeid(*a) == typeid(Dog);                    // true: the dynamic type (compiled)
typeid(*a) == typeid(Cat);                    // false
typeid(*a).name();                            // an implementation-defined name, for debugging only

if (Dog* d = dynamic_cast<Dog*>(a)) { }       // set only if *a really is a Dog (or derived from Dog)
Cat* c = dynamic_cast<Cat*>(a);               // nullptr: a Dog is not a Cat
Cat& r = dynamic_cast<Cat&>(*a);              // a reference cannot be null: throws std::bad_cast

Writer* w = dynamic_cast<Writer*>(readerPtr);  // a CROSS-cast: sideways between two bases of the same object
```

| QUESTION | ANSWER |
|---|---|
| What if the class has no virtual function? | `typeid` gives the **static** type; `dynamic_cast` down does not compile |
| Cost? | `dynamic_cast` walks the class hierarchy at run time: slower than a virtual call, avoid it on hot paths |
| `static_cast` down instead? | no check: undefined behaviour if you are wrong |
| `-fno-rtti` | compilers can turn RTTI off to save space; `typeid` and `dynamic_cast` then disappear (some embedded and low-latency code does this) |

> **A SMELL** Chains of `if (dynamic_cast<A*> ... ) else if (dynamic_cast<B*> ... )` usually mean a missing virtual function or a Visitor (More Patterns).

## Private inheritance versus composition

| KIND | MEANING | CONVERTS TO BASE OUTSIDE? |
|---|---|---|
| `public` inheritance | **is-a**: usable wherever a `Base` is | yes |
| `private` inheritance | **implemented-in-terms-of**: an implementation detail | no (compiled: `Car*` to `Engine*` is not convertible) |
| `protected` inheritance | like private, but derived classes of the derived class see the base | no |

```
struct Car : private Engine {
    void drive() { start(); }                    // uses the base internally
    using Engine::start;                         // re-export one member if you
want to
};
// Car c; Engine* e = &c;                        // ERROR: the base is private
```

- **Prefer composition** (a member `Engine engine_;`): it is simpler and does not expose the base's virtual functions to accidental overriding. Use private inheritance only to override a virtual function of the base, access a `protected` member, or get the **empty base optimisation** (an empty base takes no space; a member takes at least 1 byte).
- Never inherit publicly just to reuse code: if it is not *is-a*, it breaks the Liskov substitution principle (SOLID).

FURTHER READING

**C++ Primer (Lippman, Lajoie, Moo)**

- 18.3 Multiple and virtual inheritance (PDF pages 831 to 847)
- 19.2 Run-time type identification (PDF pages 854 to 860)

**Programming: Principles and Practice (Stroustrup)**

- 12.3 Base and derived classes (PDF pages 380 to 387)
- 12.4 to 12.5 Copying a hierarchy; benefits of OOP (PDF pages 388 to 390)

- 15.5 Destructors, including 15.5.2 Virtual destructors (PDF pages 460 to 463)